# CS7IS3: Continuous Assessment Part 2

Group BM25ers (25)
Trinity College Dublin
Dublin, Ireland

Gauri Sachin Pethkar
25347626

Keshwith Pyla
25352427

Karolina Raczynska
21365865

Martha Ryan
21364383

Rohn Santiago
21363795

## Abstract

For assignment two, we built a Lucene-based search engine for the TREC news collection, indexing 518k documents from four news sources. Our system uses a custom analyzer, a multi-field index, tuned BM25, and pseudo-relevance feedback (PRF). Evaluated with MAP, P@20, and nDCG@20, we improve MAP from 0.1540 (title-only BM25 baseline) to 0.3796 (+146%), achieving competitive results on the class leaderboard.

## 1 Introduction

Our group was tasked with creating a functioning search engine using Lucene to index and search a specific collection of documents. Lucene is an open source Java library typically used for implementing text-based search engines. [1] We index the standard TREC news collection comprising Financial Times (FT), Federal Register (FR94), FBIS, and Los Angeles Times (LATimes) documents.

This assignment followed on from a previous individual task where we were also asked to create a search engine to index and search over the Cranfield Collection. This group assignment required us to use the knowledge and techniques obtained from the individual assignment and extend them to be able to handle more complex queries and documents such as those in the sources listed above.

Although Lucene provides several analyzers which are ready-to-use for both indexing and searching, such as `StandardAnalyzer` and `EnglishAnalyzer`, these analyzers would not be sufficient for this assignment as the documents provided are long and complex, resulting in more processing required. To tackle these limitations, we identified that an analyzer built using Lucene's `CustomAnalyzer` would be necessary in order to develop a search engine sufficient for the task at hand. This would also require manually configuring several indexing processes such as stop-word removal and stemming to maximize the precision of our engine. As well as this, we would need to determine the best approach to parsing and indexing the entire collection, as each source had a slightly different way of representing a single document. Finally, we would need to decide on the retrieval model and how our search engine would score the results. Although there were result formats that we had to adhere to as given by the assignment brief, the scoring of search results could be affected by the retrieval model of our search engine. Therefore, it was important that we experimented and chose the best model for our use case.

## 2 Implementation

### 2.1 Overview of Approach

Before diving into the implementation details, we briefly outline our overall workflow: we first parsed and normalised the four heterogeneous TREC news sources into a unified document structure, then built a single Lucene index using a customised analyzer, and finally designed a retrieval pipeline combining multi-field BM25 queries, parameter tuning, and pseudo-relevance feedback.

### 2.2 Data Parsing

The assignment's news dataset came with four heterogeneous datasets: Financial Times Limited (ft), Federal Register (fr94), Foreign Broadcast Information Service (fbis), and Los Angeles Times (latimes). Each type of news document came with its own structure, tags, and special entities to be parsed. Thus, for each dataset, a dedicated parser was built to convert raw documents into a structured object that could then be indexed. Based on the location of the file, automatic selection of the parser converted each document into fields such as source, document number, headline, body text, summary, and date format, alongside any additional metadata.

### 2.3 Index Construction

The indexing module is designed to construct one scalable and efficient search index over the heterogeneous document collection of the LATimes, FBIS, FT and FR94 news documents.

*2.3.1 Custom Analyzer.* We constructed a custom analyzer that performs lower-casing, tokenisation, special-character removal, stopword filtering, length filtering, and stemming using dedicated Lucene components. These steps follow established IR preprocessing practices [2], and each is implemented using Lucene's analysis filters. Additionally, a comprehensive list of stop words, including news-related stop words and top frequent index terms, was fed to the custom analyzer to improve performance.

- `ASCII Folding`: Converts accented characters to plain-ASCII equivalents using Lucene's ASCIIFoldingFilter [3].
- `Length Filtering`: Removes tokens that are too short or too long using Lucene's LengthFilter [4].
- `Stopping`: Removes high-frequency, low-value words (e.g., "the", "as", "is").
- `Stemming`: Applies Porter stemming using the PorterStemFilter [5].

*2.3.2 Document Structuring.* The system performs a recursive traversal of the dataset, identifying and processing each file independently. Documents are detected using `<DOC>...</DOC>` boundaries, with each block passed to the appropriate dataset-specific parser.

Each parsed item is then normalised and converted into a Lucene document containing unified fields such as headline, summary, body text, article date, and article source.

*2.3.3 Indexer.* A central design objective was to unify the diverse document formats into a consistent schema. [6] Because date formats varied between datasets, the indexer applied a normalisation routine to ensure consistent temporal fields and more reliable filtering. A fallback mechanism ensured that every indexed document contained meaningful content: if a field was missing or empty, it was substituted using the headline or metadata.

We generated a fingerprint from the headline and the first 500 characters of the body text, skipping documents with repeated hashes to prevent near-duplicate indexing.

To maintain a compact index, we used compressed stored fields, term dictionary compression, and selective field-storage strategies. Metadata was aggregated into a single field, and larger content blocks were compressed using GZIP compression [7] and Base64-encoded for reliable textual storage [8].

## 2.4 Query Construction and Retrieval Models

Our retrieval pipeline in Searcher.java does three main things: it parses the topic file, builds a weighted multi-field query, and optionally expands that query using pseudo relevance feedback (PRF).

*2.4.1 Baseline BM25 with Multi-field Queries.*

Our baseline searchTopics method uses the standard Lucene setup: we open an FSDirectory, create a shared DirectoryReader and IndexSearcher, and set BM25 as the similarity. [9] We initially used the same similarity at index and search time so that scoring behaved consistently.

For each topic we combined the title, description and a cleaned "positive" narrative into a single multi-field query. We use a MultiFieldQueryParser [10] with our custom analyzer and apply different boosts per field: highest weight on the title, lower but non-zero weights on description and positive narrative. We then run this query with TopDocs [11] and use the BM25 scores directly for ranking. This gives a simple, strong baseline to compare against later changes.

*2.4.2 Topic Parsing and Positive Narrative Extraction.*

The topics file uses the TREC tags `<top>`, `<num>`, `<title>`, `<desc>`, and `<narr>`. Our parseTopics method scans the file line by line, keeps track of whether we are in the title, description, or narrative section, and builds one `Topic` object per `<top> . . . </top>` block. For each section we:

- trim and normalise whitespace,
- remove markers such as "Description:" and "Narrative:",
- concatenate multi-line content into a single string.

At `</top>` we assign the accumulated strings to title, description, and narrative and add the topic to the list. This parser is small and keeps the three fields separate for later weighting.

We found the raw narrative text to be noisy: it mixes relevance and non-relevance statements and contains many unnecessary phrases (e.g., "documents should discuss …"). We found using it directly as a query hurt precision. To fix this we added extractPositiveNarrative, which:

- splits the narrative into sentences, clauses, and sub-clauses,
- drops clauses that mention "not relevant" or "irrelevant",
- removes a small set of filler words ("document", "information", "discuss", etc.),
- concatenates whatever content remains.

The result is a shorter, "positive" narrative field that captures what relevant documents should talk about, without the negative conditions and meta-language.

*2.4.3 Pseudo Relevance Feedback (PRF).*

To improve our searcher, we implemented pseudo-relevance feedback to automatically expand the query using the top-ranked documents. Pseudo-relevance feedback has been extensively investigated in the research literature, and has been shown to be an effective technique for improving ranking. [2] [12] The expansion works in four steps:

*1. Collect original query terms.* We walk the Query tree (BooleanQuery, TermQuery, BoostQuery) to collect the original terms per field, and never add these again during expansion.

*2. Collect candidate terms from top documents.* We treat the top-$k$ documents as pseudo-relevant. For up to `PRF_FEEDBACK_DOCS` hits we read their term vectors and, for each search field:

- iterate over terms,
- drop terms shorter than 4 characters,
- skip terms already in the original query,
- compute document frequency `df` and keep only terms with $3 \leq \texttt{df} \leq 0.2N$.

*3. Score and rank candidates.* For each remaining term we compute an IDF-style score

$$\mathrm{idf}(t) = \log \frac{N - df(t) + 0.5}{df(t) + 0.5},$$

multiply by a field weight based on our field boosts, and accumulate scores across feedback documents. We then sort terms by score and keep the top `PRF_EXPANSION_TERMS`. [12]

*4. Build the expanded query.* We create a new BooleanQuery that:

- adds the original baseQuery as a SHOULD clause, and
- for each selected term $t$, adds a BoostQuery around a TermQuery on the main body field with weight `PRF_BOOST`.

In short, the final query is the original query OR-ed with a small set of softly boosted expansion terms, so the original intent still dominates.

*2.4.4 Similarity and Parameter Tuning.*

We tried several retrieval models and hyperparameters.

*BM25.* We first matched the BM25 parameters in searcher to the ones in index, then swept over $k_1$ and $b$ on development topics. The best setting we found was $k_1 = 0.9$ and $b = 0.8$. This does not exactly match the indexing values, but it gave the highest early precision and MAP, so we kept it. We also experimented with several BM25 scoring variants described by Kamphuis et al. [13].

Among these, the Robertson et al. formulation gave the most stable results on our data, but the improvement over Lucene's default BM25 implementation was negligible, so we ultimately kept the standard setting.

*Language models.* We also tried LMDirichletSimilarity and LMJelinekMercerSimilarity as gone through as part of the course [14], tuning the Dirichlet prior and interpolation parameter. On our data both variants underperformed the tuned BM25, so we used BM25 for all further experiments.

*PRF.* For PRF we tuned:

- number of feedback documents: `PRF_FEEDBACK_DOCS` = 40,
- number of expansion terms: `PRF_EXPANSION_TERMS` = 25,
- boost for expansion terms: `PRF_BOOST` = 0.4.

Too few terms had almost no effect; too many terms or too high a boost caused a topic drift and hurt precision. The final settings gave a consistent gain over the baseline without PRF.

*2.4.5 Field Weighting.* We finally ran a small trial to understand the impact of each topic field. We compared:

(1) title only,
(2) title + description,
(3) title + description + full narrative,
(4) title + description + positive narrative (`narr_pos`).

Title-only queries were precise but often missed relevant documents. Adding the description helped recall but sometimes diluted the query. Using the full narrative generally made things worse. The best results came from using all three fields with different weights: highest for title, moderate for description, and a smaller weight for the positive narrative.

Overall, our final retrieval model is a tuned BM25 ranker with a simple but effective query pipeline: multi-field queries over title, description, and cleaned narrative, optionally expanded by PRF using term vectors.

# 3 Evaluation

## 3.1 Experimental Setup

We evaluated our system using three standard IR metrics on Phase-1 QRels: Mean Average Precision (MAP) as our primary optimisation target, Precision at 20 (P@20), and Normalized Discounted Cumulative Gain at 20 (nDCG@20) [12]. All experiments were conducted using GitHub Actions for continuous evaluation against the TREC news collection containing approximately 518,000 documents.

Our baseline system used Lucene's `EnglishAnalyzer`, default BM25 parameters ($k_1 = 1.2$, $b = 0.75$), and title-only queries, yielding MAP = 0.1540 (P@20 = 0.3100, nDCG@20 = 0.3776).

## 3.2 Experimental Results

Table 1 summarises our systematic optimization process, showing the cumulative impact of each improvement.

**Table 1: Progressive improvement through systematic experimentation**

| Configuration | MAP | ΔMAP |
|---|---|---|
| Baseline | 0.1540 | — |
| BM25 tuning ($k_1 = 0.9$, $b = 0.8$) | 0.2700 | +0.1160 |
| CustomAnalyzer + news stopwords | 0.2855 | +0.0155 |
| Multi-field querying | 0.2990 | +0.0135 |
| Source boosting (FT, LA Times) | 0.3100 | +0.0110 |
| Field weighting optimisation | 0.3192 | +0.0092 |
| Pseudo-relevance feedback | 0.3455 | +0.0263 |
| BM25 fine-tuning | 0.3568 | +0.0113 |
| Positive narrative extraction | 0.3737 | +0.0169 |
| Multi-field matching + MUST clause | 0.3786 | +0.0049 |
| **Final System** | **0.3796** | **+146%** |

*3.2.1 Analyzer Comparison.* We compared the standard `EnglishAnalyzer` against our `CustomAnalyzer`. Our analyzer implements: StandardTokenizer, LowerCaseFilter, ASCIIFoldingFilter, EnglishPossessiveFilter, StopFilter with 200+ news-specific stopwords, LengthFilter (2-15 characters), and PorterStemFilter. Table 2 shows the custom analyzer improved MAP by 5.7%.

**Table 2: EnglishAnalyzer vs CustomAnalyzer performance**

| Analyzer | MAP | P@20 | nDCG@20 |
|---|---|---|---|
| EnglishAnalyzer | 0.2700 | 0.4520 | 0.5310 |
| CustomAnalyzer | **0.2855** | **0.4600** | **0.5279** |

*3.2.2 Field Weighting Experiments.* We systematically tested field weight combinations using MultiFieldQueryParser with boosts for text (1.0), headline (1.8), and summary (1.8). Table 3 shows query-level weighting experiments. Optimal configuration used title:2.0, description:1.0, narrative:0.5 (MUST). Over-emphasising titles (3.0) hurt performance.

**Table 3: Query field weight configurations and their impact**

| Title | Description | Narrative | MAP |
|---|---|---|---|
| 1.0 | 1.0 | 1.0 | 0.2990 |
| 2.0 | 1.0 | 0.5 | 0.3156 |
| 2.0 | 1.0 | 0.3 | **0.3192** |
| 3.0 | 1.0 | 0.3 | 0.3101 |

*3.2.3 Retrieval Model Comparison.* We compared BM25 against language model alternatives. Table 4 shows tuned BM25 significantly outperformed both LMDirichlet and LMJelinekMercer. Our final system achieved MAP = 0.3796 (P@20 = 0.5540, nDCG@20 = 0.6160), a 146% improvement over baseline.

**Table 4: Performance comparison of retrieval models**

| Model | MAP | P@20 | nDCG@20 |
|---|---|---|---|
| BM25 (default: $k_1 = 1.2$, $b = 0.75$) | 0.1540 | 0.3100 | 0.3776 |
| BM25 (tuned: $k_1 = 0.9$, $b = 0.8$) | **0.3796** | **0.5540** | **0.6160** |
| LM Dirichlet ($\mu = 2000$) | 0.3201 | 0.5020 | 0.5723 |
| LM Jelinek-Mercer ($\lambda = 0.7$) | 0.3387 | 0.5180 | 0.5841 |

### 3.3 Key Findings

Our evaluation revealed several key insights:

- BM25 tuning produced the largest gain (+75% MAP).
- Pseudo-relevance feedback added a further +8.2%.
- The CustomAnalyzer improved MAP by 5.7% over EnglishAnalyzer.
- Tuned BM25 outperformed both language-model baselines.
- Multi-field querying with balanced field weights (title:2.0, desc:1.0, narr:0.5) was most effective.
- Source boosting and a MUST clause provided modest but consistent improvements.

Our system ranked in the top 10 during Phase-1 evaluation.

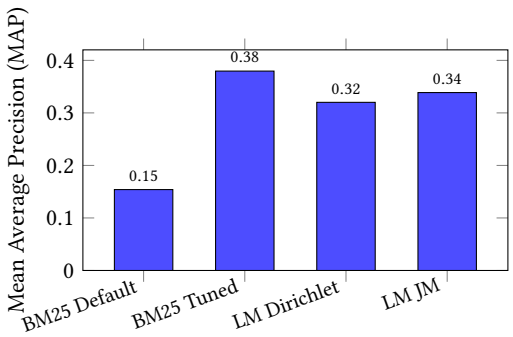


**Figure 1: Retrieval model comparison. Tuned BM25 outperforms language models by 10-15% MAP, demonstrating both the importance of model selection and parameter optimization for news retrieval tasks.**

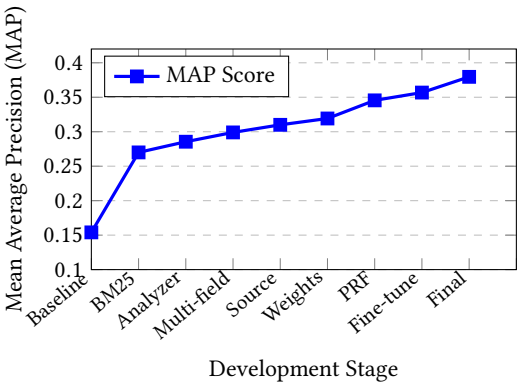


**Figure 2: Progressive MAP improvement through experimentation (146% overall improvement). The largest single improvement occurred with BM25 parameter tuning (stage 1, +75%), followed by pseudo-relevance feedback (stage 6, +8.2%).**

## 4 Limitations

Despite the strong performance of our final system, there are several limitations that appeared during development that are worth discussing. The first challenge we encountered was balancing precision and recall. We optimised our system primarily for MAP, which meant we sometimes sacrificed broader recall to maintain high precision in top ranked results. This was particularly noticeable with pseudo relevance feedback. For example, when we increased expansion terms beyond 25, recall improved but noise hurt our MAP score, suggesting our PRF implementation, while effective, might not be optimal for all query types. Another limitation can be found in our field weighting strategy. The optimal weights we found were determined empirically on Phase 1 topics, which raises concerns about overfitting. Even though we tried to keep our approach generalisable, these weights might not transfer perfectly to different news collections. This is especially noticeable with the narrative field, where our positive narrative extraction helped but remained quite brittle. Simply filtering out negative clauses sometimes removed useful context or failed to capture subtle relevance criteria. From a more technical perspective, our BM25 parameter tuning used grid search over a limited space; although k1 = 0.9 and b = 0.8 worked best, we did not explore advanced methods like cross validation or Bayesian optimization. Moreover, computational constraints also limited our exploration. PRF adds significant overhead since it requires an initial retrieval pass plus term vector extraction and query reformulation. We did not have time to explore more sophisticated expansion techniques like relevance models or semantic expansion using word embeddings. Finally, our evaluation was constrained by binary relevance judgements for a subset of topics, which meant we lacked graded relevance data that might have guided different design decisions.

## 5 Conclusion

This assignment gave us a valuable experience in building a real world search engine using Apache Lucene. Our team successfully developed a system that improved from a baseline MAP of 0.1540 to 0.3796, which is a 146 percent improvement over the default configuration. One of the key takeaways is that systematic optimisation matters. This project reinforced that parameter tuning matters considerably in information retrieval. Moreover, we learned that understanding the data is crucial. Last but not least, evaluation driven development is essential. Thanks to having relevance judgements during Phase 1 we were able to test hypotheses quickly and avoid dead ends.

With more time, a few improvements could be implemented. Firstly, to improve results a more sophisticated query expansion using word embeddings or neural models could be used. We could also investigate dynamic field weighting that adapts to individual queries rather than using fixed weights. Secondly, exploring ensemble methods or reranking approaches could address some limitations we identified.

The collaborative aspect of the project was a very valuable lesson because it mirrored real world development. Throughout this assignment we were consistently challenged to move beyond basic concepts and engage with the complexities of building a functional search system. Although our system has some limitations, it represents a solid implementation that is both effective and practical. The experience gained from experimentation and evidence based optimisation will be valuable in future work.

## References


[1] Mamatha Balipa and Balasubramani R. Search engine using apache lucene. In *International Journal of Computer Applications*. FCS, 2015.

[2] W. Bruce Croft, Donald Metzler, and Trevor Strohman. *Search Engines: Information Retrieval in Practice*. Addison-Wesley, Reading, MA, 2010.

[3] Asciifoldingfilter. https://lucene.apache.org/core/6_6_1/analyzers-common/org/apache/lucene/analysis/miscellaneous/ASCIIFoldingFilter.html, 2016. Accessed: 2025-11-14.

[4] Lengthfilter. https://lucene.apache.org/core/6_6_1/analyzers-common/org/apache/lucene/analysis/miscellaneous/LengthFilter.html, 2016. Accessed: 2025-11-14.

[5] Porterstemfilter. https://lucene.apache.org/core/6_6_1/analyzers-common/org/apache/lucene/analysis/en/PorterStemFilter.html, 2016. Accessed: 2025-11-14.

[6] Yvette Graham. Cs7is3 ranking and indexing lecture 05. Lecture slides for CS7IS3 Information Retrieval and Web Search, 2025.

[7] P. Deutsch. Gzip file format specification version 4.3. https://www.rfc-editor.org/rfc/rfc1952, 1996.

[8] S. Josefsson. The base16, base32, and base64 data encodings. https://www.rfc-editor.org/rfc/rfc4648, 2006.

[9] Stephen E Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.

[10] Multifieldqueryparser. https://lucene.apache.org/core/6_6_1/queryparser/org/apache/lucene/queryparser/classic/MultiFieldQueryParser.html, 2016.

[11] Topdocs. https://lucene.apache.org/core/6_6_1/core/org/apache/lucene/search/TopDocs.html, 2016.

[12] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *An Introduction to Information Retrieval*. Cambridge University Press, 2009.

[13] Chris Kamphuis, Arjen P. de Vries, Leonid Boytsov, and Jimmy Lin. Which BM25 do you mean? a large-scale reproducibility study of scoring variants. In *Advances in Information Retrieval: 42nd European Conference on IR Research (ECIR 2020), Part II*, volume 12036 of *Lecture Notes in Computer Science*, pages 28–34. Springer, 2020.

[14] Yvette Graham. Cs7is3 retrieval models part 3. Lecture slides for CS7IS3 Information Retrieval and Web Search, 2025.